

TEST ASSESSMENT

Q1) Write a function to find the longest common prefix string amongst an array of string.

If there is no common prefix, return an empty string "".

Java code :

class Solution {

public String longestCommonPrefix(String[] args) {

if (args == null || args.length == 0) {

return "";

}

String prefix = args[0];

for (int i = 1; i < args.length; i++) {

if (args[i] == null) {

return "";

}

while (!args[i].startsWith(prefix)) {

prefix = prefix.substring(0, prefix.length() - 1);

if (prefix.length() == 0) {

return "";

}

}

}

return prefix;

}

}

Example 1 :

Input strs = ["flower", "flow", "flight"]

output = "fl"

Example 2 :

Input strs = ["dog", "racecar", "car"]

output = ""

```
public static void main (String args[]) {
```

```
    Scanner sc = new Scanner (System.in);
```

```
    int n = sc.nextInt();
```

```
    String[] str = new String[n];
```

```
    for (int i = 0; i < n; i++) {
```

```
        str[i] = sc.next();
```

```
    }
```

```
    Solution obj = new Solution();
```

```
    String result = obj.longestCommonPrefix (str);
```

```
    System.out.println (result);
```

```
}
```

```
}
```

Q2)

You are given a 0-indexed array of integers `nums` of length `n`. You are initially positioned at index 0.

Each element `nums[i]` represents the maximum length of forward jump from index `i`.

$$0 \leq j \leq \text{nums}[i] \quad ?$$

$$i + j < n$$

Return the minimum number of jumps.

Java code:

class Solution {

public int jump(int[] nums) {

int jump = 0;

int end = 0;

int farthest = 0;

for (int i = 0; i < nums.length - 1; i++)

{

farthest = Math.min(farthest, i + nums[i]);

if (i == end) {

jump++;

end = farthest;

}

}

return jump;

}

}

Example 1:

Input `nums = [2,3,1,1,4]`

Output: 2.

Example 2:

Input `nums = [2,3,0,1,4]`

Output: 2.

```
public static void main(String[] args) {  
    Scanner sc = new Scanner(System.in);  
    int n = sc.nextInt();  
    int[] nums = new int[n];  
    for (int i = 0; i < n; i++) {  
        nums[i] = sc.nextInt();  
    }  
    Solution obj = new Solution();  
    int result = obj.jump(nums);  
    System.out.println(result);  
}
```